



AI and Code: Two Projects from Next

Don Syme (dsyme@github.com) + all of GitHub Next
Principal Researcher, GitHub Next





GitHub Next

Researching the future of
software development

githubnext.com



What is GitHub Next?

What

An applied R&D group attached to GitHub, reports to Thomas

Mission

Transform the practice of software development

Mode of Operation

Build, Release, Learn, Co-operate.

Who

~15 applied LLM/ML experts (many ex-Copilot), UX experts, CS experts

Why this is the right way to run innovative applied R&D

Operates at the Goldilocks distance!



Released March 23!

Copilot for Docs

March 22, 2023

WAITLIST

How would it feel to have an expert on hand at all times? We built a tool that uses relevant information from a project's documentation to answer questions or explain concepts.

Copilot for Pull Requests

March 22, 2023

WAITLIST

Pull requests are a central part of the GitHub user experience. Copilot for PRs brings the power of Copilot to the PR experience, to help you write better PR descriptions, and to help your team review and merge PRs faster.

Copilot for CLI

March 22, 2023

WAITLIST

Ever having trouble remembering that shell command or this obscure flag? Don't worry: we're building GitHub Copilot assistance right into your terminal

Copilot Voice

March 22, 2023

WAITLIST

Write code without the keyboard. Difficulty typing? Use your voice to code without spelling things out by talking with GitHub Copilot.



Today's Menu

Equipping GPT-4 with Numeric Calculation (clone)

Interlude

Pervasive Copilot summarization in GitHub

(aside: I help guide F# lang design ~10-50%)



Part 1 - Equipping GPT-4 with Numeric Calculation



caveat

AI is moving unbelievably fast. This work was done in ancient times, two weeks ago, before the ChatGPT “Code Interpreter” add-in was released

More on that in the interlude



Equipping for Numeric Calculation

The situation:

- Any GPT-4 Chat: Context + Question $\Rightarrow$ Answer



Equipping for Numeric Calculation

The situation:

- Any GPT-4 Chat: Context + Question $\Rightarrow$ Answer

The problem:

- GPT-4 is **utterly, disastrously, catastrophically** terrible at numeric calculations
- It's also terrible at numeric comparisons

*GPT-4 should not be trusted to write a number that is not present verbatim in the input, nor to reason about numbers in any significant way. **In trust scenarios, don't allow GPT-4 to write numbers, and beware that every numeric comparison may be flawed.***



How bad is this?

Very very bad



Example fails

- Bing AI can't be trusted
- Example garbage:

Both companies reported an increase in net sales compared to the same quarter last year, but Gap Inc. had a much larger absolute and ~~the~~ increase (\$4.04 billion, up 2%) than lululemon (\$1.9 billion, up 28%).



More example garbage:

Solving for n , we get:

$$n = \ln(2) / \ln(1 + 0.3)$$

Using a ~~calculator~~ calculator, we get:

$$n \approx 2.53$$



Therefore, it takes about 2.53 years for an item growing at 30% annually to double.



The Hope

The observation:

👎 GPT-4 is **terrible** at **numeric calculations**

👍 GPT-4 is **good** at writing **numeric calculation code**



The Hope made Real

Without numeric calculation equipping:

1. **GPT-4:** Question --> Answer

With numeric calculation equipping:

1. **GPT-4:** Question --> Numeric calculation code
2. **Evaluator:** Numeric calculation code --> Numeric calculation answers
3. **GPT-4:** Question + Numeric calculation code + Numeric calculation answers --> Answer

My opinion: this two-phase technique should be incorporated into **every** GPT-4 request that involves numbers unless explicitly known to be calculation-free. Anything else is



Example question

This is an extract from the Microsoft annual report for 2022:

Microsoft repurchased the following shares of common stock under the share repurchase programs:

(In millions)	Shares	Amount	Shares	Amount	Shares	Amount
Year Ended June 30,		2022		2021		2020
First Quarter	21	\$6,200	25	\$5,270	29	\$4,000
Second Quarter	20	6,233	27	5,750	32	4,600
Third Quarter	26	7,800	25	5,750	37	6,000
Fourth Quarter	28	7,800	24	6,200	28	5,088

What is the overall average and standard deviation of shares repurchased per quarter over these three years in dollars?



Example fail

...lots of working ...

Therefore, the overall average and standard deviation of shares repurchased per quarter over these three years in dollars are \$5 million dollars and 1.16 million dollars, respectively.



Example prompt

Guidance

Do not answer the question. Instead, your task is to write some numeric calculation and comparisons relevant to answering the question.

After the question write a code block with up to three sections containing content relevant to answering the question.

In the "Definitions" section define a label for each number in the original question like `car\_count` or `speed\_of\_car\_in\_km\_per\_hour`.

- * Every label name should include the unit of measure if known.
- * This section should be valid Python and can include valid Python single-dimensional arrays.
- * Do not use or create multi-dimensional arrays.
- * Give each label a unit of measure in a comment after each definition.
- * Document the meaning of each definition in the comment.
- * If the unit of measure is unknown use "unknown".
- * Omit this section if there are no numbers in the question.

In the "Calculations" section define additional relevant labels using



Example relevant calculation code

```
# Definitions
shares_2022 = [21, 20, 26, 28] # millions of shares
amount_2022 = [6200, 6233, 7800, 7800] # millions of dollars
shares_2021 = [25, 27, 25, 24] # millions of shares
amount_2021 = [5270, 5750, 5750, 6200] # millions of dollars
shares_2020 = [29, 32, 37, 28] # millions of shares
amount_2020 = [4000, 4600, 6000, 5088] # millions of dollars

# Calculations
total_shares = shares_2022 + shares_2021 + shares_2020 # millions of shares
total_amount = amount_2022 + amount_2021 + amount_2020 # millions of dollars
average_amount = np.mean(total_amount) # millions of dollars
std_amount = np.std(total_amount) # millions of dollars
```



Example relevant results

```
total_shares [21, 20, 26, 28, 25, 27, 25, 24, 29, 32, 37, 28]  
total_amount [6200, 6233, 7800, 7800, 5270, 5750, 5750, 6200, 4000, 4600, 6000, 5088]  
more_shares_2022 False  
more_amount_2022 True  
average_amount 5890.916666666667  
std_amount 1077.077175997874
```

Add code + results to the prompt, before the question



Example answer

The overall average and standard deviation of shares repurchased per quarter over these three years in dollars are:

- Average: \$5,890.92 million
- Standard deviation: \$1,077.08 million

...lots of working ...

Therefore, the overall average and standard deviation of shares repurchased per quarter over these three years in dollars are \$5,890.92 million dollars and \$1,077.08 million dollars, respectively.



Effective?

Raw calculation (decimals, integers, math functions):

- without equip: **50% mistake rate** (77/153), many extreme mistakes
- with equip: **0% mistake rate** (153/153)

Financial calculations (mortgage, rate of return, WACC):

- without equip: **57% mistake rate**, 4/14
- with equip: **14% mistake rate** 12/14

Tabular data calculations:

- without equip: **50% mistake rate** 4/8
- with equip: **0% mistake rate** 8/8

Unit conversion calculations problems

- without equip: **36% mistake rate** 7/11
- with equip: **0% mistake rate** 11/11

Date calculations:

- without equip: **30% mistake rate** 7/10
- with equip: **20% mistake rate** 8/10

Word puzzles:

- Without equip: **11% mistake rate** (253 failures 2050/2303)
- With equip: **6.2% mistake rate** (145 failures 2158/2303)



On Explanation

The industry is crying out for AI that can be *trusted*, can be *explained*, can generate *trustworthy, recognisable, analyzable* artifacts like *spreadsheets* and *notebooks*.

Imagine this UX: hover over a number and you see the relevant calculation, and can get a spreadsheet or notebook for it.

By generating calculation code, we have a stepping stone to this.



Interlude - Welcome to the Future



Some mantras

Calculation is hard (for AI). Writing calculation code is easy.

Computation is hard (for AI). Writing code is easy.

("Hey AI bro, you need that code after all?")



Welcome to the Future

This category of technique can be applied to **any language, computational logic, DSL, query language, library, service or computational capability.**

The outputs can be **answers, code, notebooks, spreadsheets, projects, tests, documentation.**

The outputs can be **generalized and deployed.**



Welcome to the Future

Question $\Rightarrow$ Answer



Welcome to the Future

Documentation + Description $\Rightarrow$ Code

Code $\Rightarrow$ Results

Documentation + Description + Results $\Rightarrow$ Outputs

(AI can also iterate, plan, correct, generate tests)



“Code Interpreter” ChatGPT AddIn

Code interpreter Alpha

An experimental ChatGPT model that can use

We p  Ethan Mollick
@emollick

envi This was kind of delightful: I uploaded a CSV file of every lighthouse
location in the US.

eval "ChatGPT Code Interpreter: Create a gif of a map of the lighthouse
locations, where the map is very dark but each lighthouse twinkles." A
uppe couple seconds later...

uplo



|| GIF

Boost Your Data Workflow with Noteable and ChatGPT

ChatGPT Plugin for Jupyter Notebooks

Get Started





a brick weighs 30% of a brick plus five GPUs there are 100 bricks and 200 GPUs in a metric ton how much does a brick weigh?



a brick weighs 30% of a brick plus five GPUs there are 100 bricks and 200 GPUs in a metric ton how much does a brick weigh?



Let's call the weight of a brick "B" and the weight of a GPU "G". We can set up an equation based on the information given:

$$B = 0.3 * B + 5 * G$$

We can also set up another equation based on the total weight of 100 bricks and 200 GPUs being equal to 1 metric ton (1000 kilograms):

$$100 * B + 200 * G = 1000$$

We can solve this system of equations to find the weight of a brick (B) and the weight of a GPU (G).

Finished calculating ▾

The weight of a brick is 7.8125 kg.

```
from sympy import symbols, Eq, solve
```

```
B, G = symbols('B G')  
eq1 = Eq(B, 0.3 * B + 5 * G)  
eq2 = Eq(100 * B + 200 * G, 1000)  
solution = solve((eq1, eq2), (B, G))  
solution[B]
```

```
# Result  
7.812500000000000
```



Welcome to the Future

The integration of **Chat, Data and Programming** is coming very fast. Much faster than I expected even a month ago.

We will now see an **avalanche of ChatGPT addins**. An **app-like ecosystem** will develop very rapidly, where addins have access to **compute, storage, indexed documentation, networking, data resources, UX** and more.

Important: chat is not just one thing:

- Chat for Entertainment
- Chat for Search
- Chat for Analytical Question Answering
- Chat for Technical Creation



Part 2 - Infusing github.com with AI Summaries

github.com/githubnext/prbot
githubnext.com/projects/copilot-for-pull-requests



Summarize all the things

Code $\Rightarrow$ Summary (“Explain this code” in Copilot VS Code)

PR $\Rightarrow$ Summary (Copilot for PRs)

File $\Rightarrow$ Summary

Directory $\Rightarrow$ Summary

Issue $\Rightarrow$ Summary

Discussion $\Rightarrow$ Summary

Repo $\Rightarrow$ Summary

(multiple AI calls needed == hierarchical summarization)



Example – PR Summary

`./prbot describe https://github.com/githubnext/prbot/pull/565`

▼ Summary

This pull request enhances the bot's ability to describe code changes, updates the test scores for the describe task, and simplifies the poetry generation code. It modifies `src/bot/bot-describe.ts` to improve the description quality and readability, updates `test/scores/boosted/describe-score.md` to report the latest results, and removes an unused case from `src/engine/poems.ts` to improve code clarity.



Sample short summaries for files

 scorer	<i>Scripts and data for evaluating prbot summaries and walkthroughs.</i>	2 weeks ago
 src	<i>Source code for prbot, a GitHub bot using OpenAI models.</i>	3 hours ago
 test	<i>Tests, baselines, and scores for the prbot tool.</i>	4 hours ago
 .dockerignore	<i>Files and directories to exclude from Docker build context.</i>	6 months ago
 .env.example	<i>Environment variables for prbot, including credentials and proxy URL.</i>	4 months ago
 .eslintignore	<i>Files and directories to exclude from ESLint.</i>	4 months ago
 .eslintrc	<i>ESLint configuration for TypeScript files with plugins and rules.</i>	4 months ago



Hierarchical summaries of entire repositories

Overall summary of 1M+ LOC code in github/github “app” directory.

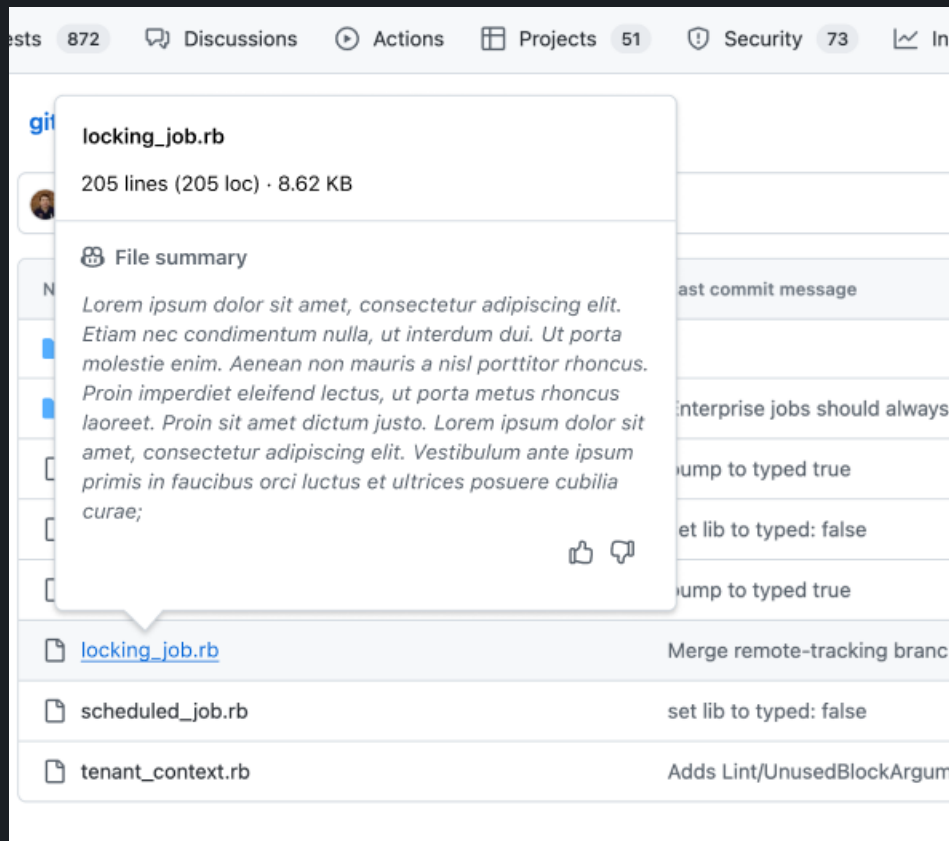
Every chunk, file, directory, issue, discussion gets a summary too.

The directory `app/` contains files and subdirectories for the [GitHub](#) web application, which allows users to create, manage, and collaborate on software projects.

- `app/api/`
 - The [GitHub API](#), which provides programmatic access to GitHub features and data.
 - Includes core API files, serializers, and subdirectories for various features such as organization, repository, code scanning, team, search, runtime, staff tools, enterprise, codespaces, deployments, and environment.
- `app/components/`
 - View components for various features and pages of the GitHub web application.
 - Uses the [view_component](#) library and the [sorbet](#) type checker.
 - Includes components for account verifications, GitHub Actions, GitHub Advanced Security, security advisories, animation, apps, beta features, billing, blobs, branch, businesses, checks, closables, code navigation, code scanning, codespaces, command palette, comments, commits, compare, compliance, conditional access, conduit, config as code, context switcher, copilot, dashboard, dependabot, dependency graph and review, devtools, diffs, discussions, environments, events, explore, feed, feed posts, files, fragments, flipper features, gists, and hovercards.
- `app/controllers/`



Some future UX (from github designers)



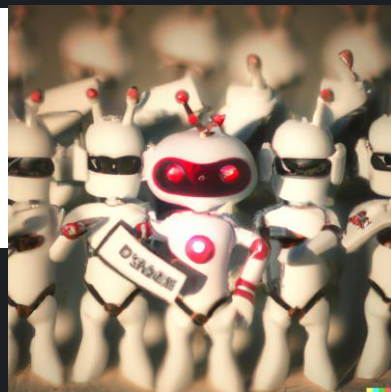
A bit of fun – AI Poetry for PRs

*We unleash the prbot, the beast of AI
We measure its power, its skill and its price
We refine its language, its summaries and scores
We challenge the baseline, the human and the norms*

*Command line `prbot-engine`
Added for automation ease
Springtime coding bliss `#!`*

*No mercy for the bots, they cannot fool us
We log and return, we don't let them rule us
We parse the commands, we don't need the labels
We test the payloads, we are the rebels*

*Oh, we're the crew who code the bot, and we're the best you'll ever see
We review the changes in the files, with a `fileIndex` for each
We heave and ho and pull the rope, and make the locators neat
We're the crew who code the bot, and we're the pride of the fleet*



A bit of fun – AI Stories for Repos

Based on overall summary of all 1M LOC code in github/github “app” directory

SAGAN: *The `app/` directory is a microcosm of the human condition, a mirror of our strengths and weaknesses, our hopes and fears, our dreams and realities. It is a reminder of how far we have come, and how much further we can go....*

ATTENBOROUGH: *In the vast and diverse ecosystem of GitHub, there is a hidden world of code that powers the web application millions of users rely on every day. This world is the app directory, where we can find files and subdirectories for various features and services that GitHub offers. Let's take a closer look at some of the inhabitants of this directory, and see how they interact and evolve.*

MACHIAVELLI: *To the magnificent and generous prince, who has graciously bestowed upon me the honor of examining the inner workings of his most esteemed and powerful enterprise, I dedicate this humble treatise on the directory `app/` in the repository `github/github`. ...the gateway to the prince's vast dominion of code and collaboration. ..*





Thank you!

dsyme@github.com



Context

The big game is using LLMs to bring **assists** to **dev activities**.

Many possible **assists**

Many different **activities**

Delivery in **editor**, **monolith**, **application**, **command line** and **new modalities**

LLMs are **probabilistic text calculators**

Text in, text out.

Key topics: **prompts**, **context**, **control**, **quality**, **divide-and-conquer**, **chain-of-thought**, **hallucination**.

LLMs are **magical** but **difficult** functions to integrate in **high-trust scenarios**.

Scepticism is healthy. Almost nothing in github.com today is probabilistic.

The human is almost certainly in-the-loop.



AI Outputs Overview

Summarization

Indexing

Chat

Generative

Critique



AI Outputs Overview

Summarization

- Keywords
- Title
- Abstract
- Walkthrough
- Documentation

Indexing

Chat

Generative

Critique



Summarization: Rationale

We are headed to a world where **most text associated with repos** is **emergent** and **automatic**.

Content $\Rightarrow$ Search

Content $\Rightarrow$ Text

Uses:

- Abstracts, keywords, descriptions
- Table of contents
navigation
- Documentation, walkthroughs

= for comprehension
= for
= for explanation

Use cases:

- Tool-tips
Infopanel content
- Seeding human-authored descriptions

Search



Esp. when **unfamiliar** with content, **pressed for time**, or **small-factor devices**

Summarization: Technique

PR Chunks + some context

(GPT-4) → Descriptions of Chunks

(GPT-4) → Summaries of descriptions of chunks

(GPT-4) → Final outputs



Some Principles of AI UX

Utility

Quality

Trust

Handling Change



Some Principles of AI UX

Utility

- The AI outputs must make some human task faster, or better
- All new outputs take user attention. They aren't free

Quality

Trust

Handling Change



Some Principles of AI UX

Utility

Quality

- The AI makes mistakes, must be minimized
- Small inputs + Longer outputs → More mistakes
- The user must edit, approve/reject or tolerate
- Users are more tolerant of mistakes in ephemeral outputs

Trust

Handling Change



Some Principles of AI UX

Utility

Quality

Trust

- Mistakes are highly corrosive
- Authorship must be understood
- A human can explicitly take ownership after checking

Handling Change



Some Principles of AI UX

Utility

Quality

Trust

Handling Change

- Outputs should be time-labelled, fresh and up-to-date
- User-editing of outputs conflicts with freshness



Video of Copilot for PRs

